

Deadlocks solutions using Semaphore/Dining Philosopher

Solution:1

Allow at most **4 philosophers** to be at the table at once. This guarantees at least one philosopher can get both forks.

```
c
#include <semaphore.h>
#include <pthread.h>

#define N 5 // philosophers

sem_t forks[N];          // binary semaphores for forks
sem_t room;              // counting semaphore for table capacity

void philosopher(int i) {
    int left = i;
    int right = (i + 1) % N;

    while (1) {
        // think
        sem_wait(&room);          // wait for a seat (max 4)
        sem_wait(&forks[left]); // pick up left fork
        sem_wait(&forks[right]); // pick up right fork
        // eat
        sem_post(&forks[right]);
        sem_post(&forks[left]);
        sem_post(&room);          // Leave seat
        // think again
    }
}

int main() {
    for (int i = 0; i < N; i++)
        sem_init(&forks[i], 0, 1); // each fork is available

    sem_init(&room, 0, N-1);        // only N-1 philosophers allowed

    pthread_t philosophers[N];
    for (int i = 0; i < N; i++)
        pthread_create(&philosophers[i], NULL, philosopher, i);
    // ...
}
```

Solution 2: Asymmetric pickup order (odd/even rule)

- Even-numbered philosophers pick up left then right.
- Odd-numbered philosophers pick up right then left.

```
void philosopher(int i) {
    int first, second;
    if (i % 2 == 0) {
        first = i;           // left
        second = (i + 1) % N; // right
    } else {
        first = (i + 1) % N; // right
        second = i;          // left
    }

    while (1) {
        sem_wait(&forks[first]);
        sem_wait(&forks[second]);
        // eat
        sem_post(&forks[second]);
        sem_post(&forks[first]);
    }
}
```

Solution 3: Use try-wait (non-blocking acquisition)

```
void acquire_two(sem_t *s1, sem_t *s2) {
    while (1) {
        wait(s1);           // (1) acquire first lock (blocks if needed)
        if (sem_trywait(s2) == 0) { // (2) try to get second lock
            // acquired both successfully
            return;          // (3) exit function - both locks held
        }
        // failed to get s2
        signal(s1);          // (4) release first lock immediately
        // optional: random backoff or yield
    }
    // (5) loop and try again
}
```